Model Order Reduction Techniques

Miquel P. Baztán Grau
Supervisor: Dr. Matteo Giacomini

September 8, 2025

Contents

1 Introduction

The numerical simulation of complex physical systems often involves the solution of high-dimensional models, typically derived from discretizations of partial differential equations (PDEs). While these full-order models (FOMs) can achieve high fidelity, they are computationally expensive in terms of time and memory, especially when used in contexts that require repeated evaluations, such as optimization, uncertainty quantification, parameter estimation, or real-time control. This computational burden motivates the development of **model order reduction (MOR)** techniques.

Model order reduction aims to construct a low-dimensional surrogate, known as a reduced-order model (ROM), that accurately reproduces the essential dynamics of the full system while dramatically reducing computational cost. This is generally achieved by projecting the governing equations onto a reduced basis extracted from the high-dimensional solution space, or by employing data-driven approaches that exploit patterns in simulation or experimental data.

The key objective of MOR is to balance efficiency and accuracy: reduced-order models should retain the predictive power of the original model, while being lightweight enough to enable applications that would otherwise be infeasible. Over the past decades, MOR has become a central tool in applied mathematics, computational science, and engineering, with successful applications in fields such as fluid dynamics, structural mechanics, electromagnetics, and energy systems.

2 Dataset description

Suppose we have a $R = R_x \times R_y$ rectangle and we apply a unitary force to the slice $L_t = L_{x_t} \times L_{y_t}$ with an angle θ_k, where $t = 1, ..., T$ and $k = 1, ..., K$.

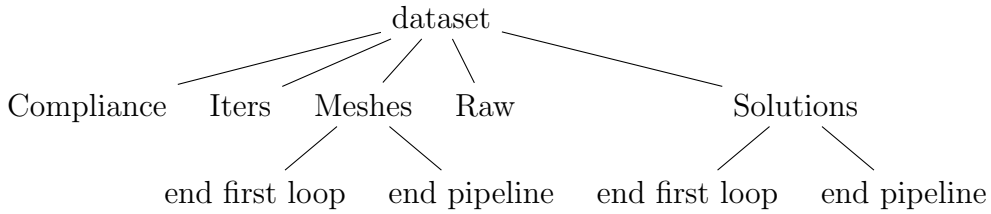
The problem of topology optimisation is defined as follows:

$$\begin{aligned} \min_{\Omega \in \mathcal{U}_{ad}} \quad & J(\Omega, \boldsymbol{u}) = \int_{\Gamma_N} \boldsymbol{g} \cdot \boldsymbol{u} \, ds \\ \text{s.t.} \quad & -\boldsymbol{\nabla} \cdot \boldsymbol{\sigma}(\boldsymbol{u}) = \boldsymbol{0} \quad \text{in } \Omega, \\ & \boldsymbol{u} = \boldsymbol{0} \quad \text{on } \Gamma_D, \\ & \boldsymbol{\sigma}(\boldsymbol{u})\boldsymbol{n} = \boldsymbol{g} \quad \text{on } \Gamma_N, \\ & \boldsymbol{\sigma}(\boldsymbol{u})\boldsymbol{n} = \boldsymbol{0} \quad \text{on } \Gamma_F, \\ & \int_{\Omega} 1 \, d\boldsymbol{x} \leq V^\star. \end{aligned} \tag{1}$$

For each pair (L_t, θ_k) we solve this problem via standard finite element methods and using the framework FreeFEM++, which is highly used for solving PDEs. The solver is composed of two loops: the first one is responsible for making a initial approximation of the solution (which usually is quite blur), whilst the second one aims to improve

that first approximation and modifies the mesh to an adapted one (which performs better).

Our dataset folder is then organized as follows:



Hence, for each snapshot we have (in order of appearance in the tree):

- The compliance computed each iteration of the optimization pipeline
- How many iterations we need for both the first and last loop
- The final meshes of both loops
- The solution within a structured mesh
- The final solutions of both loops

We also have some scripts for doing miscellaneous computations and the structured mesh.

We will denote the (L_t, θ_k)-snapshot computed in the *Raw* folder by $\Phi_{t,k} \in [0,1]^D$ (0 no material, 1 concentrated material). Sometimes we will refer $\boldsymbol{\mu} = (\mu_1, \mu_2) = (t, k)$ to be the parameters of the model.

In our dataset we use the following parameters:

Params	**Description**	**Value**
R	Rectangle size	$[0,2] \times [0,1]$
L_t	Force application slice	$\begin{cases} [\frac{15+t}{16}, \frac{16+t}{16}] \times \{0\}, \ t \in [1,16] \\ \{2\} \times [\frac{t-17}{16}, \frac{t-16}{16}], \ t \in [17,32] \\ [\frac{64-t}{16}, \frac{65-t}{16}] \times \{1\}, \ t \in [33,48] \end{cases}$
θ_k	Angle of the force	$\frac{\pi}{90} + \frac{\pi}{36}(k-1)$
(T, K)	Number of snapshots	$(48, 36)$
D	Snapshot size	10011

Table 1: Parameters for the model used

3 Proper Orthogonal Decomposition (POD)

Among the various model order reduction techniques, *Proper Orthogonal Decomposition (POD)* has emerged as one of the most widely used and effective approaches. POD is a projection-based method that identifies a set of orthogonal modes capable of capturing the most energetic or dominant features of a high-dimensional system. These modes are extracted directly from solution data—often referred to as *snapshots*—obtained by simulating the full-order model under representative conditions.

Mathematically, let $\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_m \in \mathbb{R}^N$ denote a set of m snapshots of the full-order solution, where N is the dimension of the high-fidelity model. The snapshot matrix is defined as

$$\mathbf{U} = [\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_m] \in \mathbb{R}^{N \times m}. \tag{2}$$

POD constructs an orthonormal basis $\{\boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_r\}$, with $r \ll N$, by solving the eigenvalue problem

$$\mathbf{C}\boldsymbol{\phi}_i = \lambda_i \boldsymbol{\phi}_i, \quad i = 1, \ldots, r, \tag{3}$$

where $\mathbf{C} = \mathbf{U}\mathbf{U}^\top$ is the correlation matrix of the snapshots and λ_i are the eigenvalues sorted in descending order. Equivalently, the basis can be computed using a singular value decomposition (SVD) of the snapshot matrix:

$$\mathbf{U} = \mathbf{W}\boldsymbol{\Sigma}\mathbf{V}^\top, \tag{4}$$

where the first r columns of $\mathbf{W}$ form the POD basis. The reduced-order solution is then approximated as

$$\mathbf{u} \approx \sum_{i=1}^{r} a_i(t)\, \boldsymbol{\phi}_i, \tag{5}$$

with time-dependent coefficients $a_i(t)$ obtained, for instance, via Galerkin projection of the governing equations onto the reduced basis.

The primary advantage of POD lies in its ability to produce a low-dimensional approximation that retains the dominant dynamics of the system. By focusing on the most energetic modes, POD-based reduced-order models achieve significant computational savings while maintaining high accuracy, making them particularly suitable for tasks such as real-time simulation, optimization, and uncertainty quantification across a wide range of scientific and engineering applications.

When dealing with multidimensional data, such as parameterized or time-dependent fields, it is often more natural to represent the dataset as a tensor rather than flattening it into a matrix. To extend the Proper Orthogonal Decomposition (POD) framework to tensors, we rely on the Tucker decomposition, also known as the Higher-Order Singular Value Decomposition (HOSVD).

The Tucker decomposition generalizes the matrix SVD to higher-order tensors by factorizing a tensor into a core tensor and a set of orthogonal mode matrices, one for each dimension (or mode) of the data. This allows us to extract low-dimensional structures while preserving the multi-way nature of the data.

Formally, given a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$, the HOSVD expresses it as

$$\mathcal{X} \approx \mathcal{G} \times_1 U^{(1)} \times_2 U^{(2)} \cdots \times_N U^{(N)}$$

where $\mathcal{G}$ is the core tensor and $U^{(n)}$ are orthogonal factor matrices obtained from the SVD of the mode-n unfoldings of $\mathcal{X}$.

This decomposition provides the natural extension of POD to tensors: instead of finding dominant modes in a single matrix, we identify dominant modes along each dimension of the tensor, leading to a more compact and interpretable reduced representation.

3.1 Reduction

We consider the $T \times K$ snapshots stored in the *Raw* folder and assemble them into the 3D-tensor $\mathcal{A} \in \mathbb{R}^{T\times K\times D}$ where the data are centered as follows:

$$\mathcal{A}_{t,k} = \begin{pmatrix} | \\ \hat{\Phi}_{t,k} \\ | \end{pmatrix}, \quad \hat{\Phi}_{t,k} = \Phi_{t,k} - \bar{\Phi}, \quad \bar{\Phi} = \frac{1}{TK}\sum_{t=1}^{T}\sum_{k=1}^{K}\Phi_{t,k}. \tag{6}$$

We perform the HOSVD to the tensor. That is $\mathcal{A} = \mathcal{M} \times_1 \mathbf{U} \times_2 \mathbf{V} \times_3 \mathbf{W}$. Given $\boldsymbol{r} = (r_1, r_2, r_3) \in \mathbb{N}^3$, we define $\mathcal{A}_{\boldsymbol{r}} = \mathcal{M}_{\boldsymbol{r}} \times_1 \mathbf{U} \times_2 \mathbf{V} \times_3 \mathbf{W}$, to be the approximation of the tensor $\mathcal{A}$ with rank $\boldsymbol{r}$, where we set $m_{ijl} = 0, \forall i > r_1, \forall j > r_2, \forall l > r_3$. We denote $\hat{\Phi}_{t,k}^{POD_{\boldsymbol{r}}}$ the $(t,k)^{th}$ column of $\mathcal{A}$ and say that $\hat{\Phi}_{t,k}^{POD_{\boldsymbol{r}}}$ is the approximation of the snapshot (t,k) with rank $\boldsymbol{r}$.

We define two approximation errors:

$$\hat{e}_{\boldsymbol{r}} = \max_{\substack{t=1,\dots,T\\k=1,\dots,K}} \left\{ \frac{\|\hat{\Phi}_{t,k} - \hat{\Phi}_{t,k}^{POD_{\boldsymbol{r}}}\|_2}{\|\hat{\Phi}_{t,k}\|_2} \right\}, \qquad e_{\boldsymbol{r}} = \max_{\substack{t=1,\dots,T\\k=1,\dots,K}} \left\{ \frac{\|\Phi_{t,k} - \Phi_{t,k}^{POD_{\boldsymbol{r}}}\|_2}{\|\Phi_{t,k}\|_2} \right\}$$

By equation (6), $\Phi_{t,k} = \bar{\Phi} + \hat{\Phi}_{t,k}$ and thus $\Phi_{t,k}^{POD_{\boldsymbol{r}}} = \bar{\Phi} + \hat{\Phi}_{t,k}^{POD_{\boldsymbol{r}}}$. We will say that $\hat{e}_{\boldsymbol{r}}$ is the centered snapshot error and $e_{\boldsymbol{r}}$ is the snapshot error.

We choose the optimal rank value $\boldsymbol{r}^* = (r_1^*, r_2^*, r_3^*)$ using the following criteria.

Given $\mathcal{T} \in \mathbb{R}^{n_1\times n_2\times n_3}$, we define the i^{th} unfold of $\mathcal{T}$ to be the matrix $\pi_i(\mathcal{T}) \in \mathbb{R}^{n_i\times(n_j\cdot n_k)}$, where $\{n_i, n_j, n_k\} = \{n_1, n_2, n_3\}$. That is, we slice the tensor $\mathcal{T}$ and join the matrices keeping the first dimension of the matrix the same as the i^{th} dimension of the original tensor.

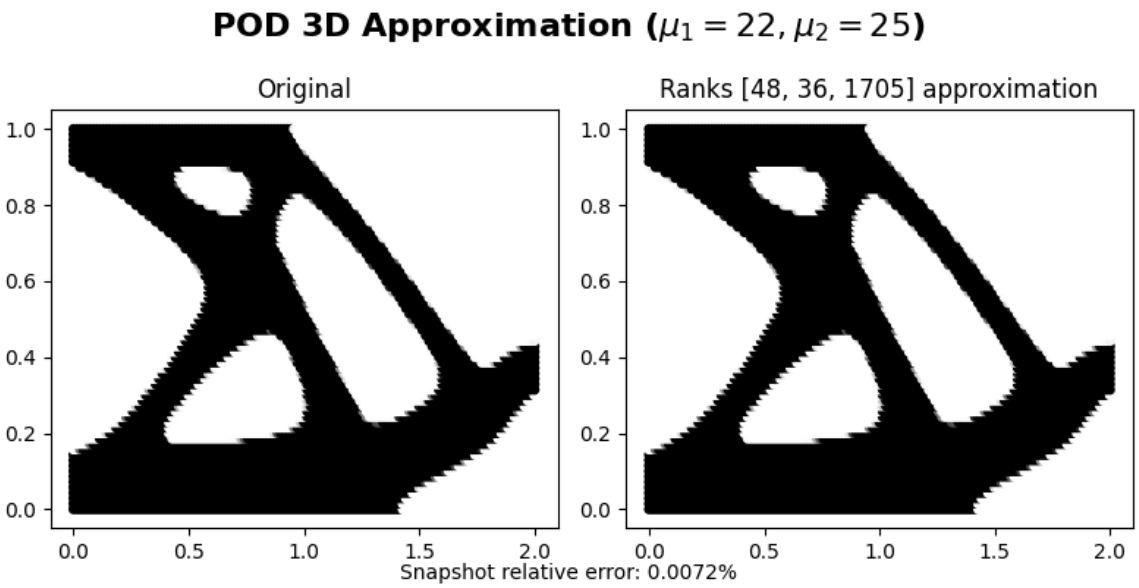
Then, given $\epsilon > 0$, r_i^* is the minimum r satisfying $\sum_{j=1}^{r}\sigma_j \geq (1-\epsilon)\sum_{j=1}^{M}\sigma_j$, where σ_j are the singular values of $\pi_i(\mathcal{A})$ such that $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_M \geq 0$, where $M = \min\{n_i, n_j\cdot n_k\}$.

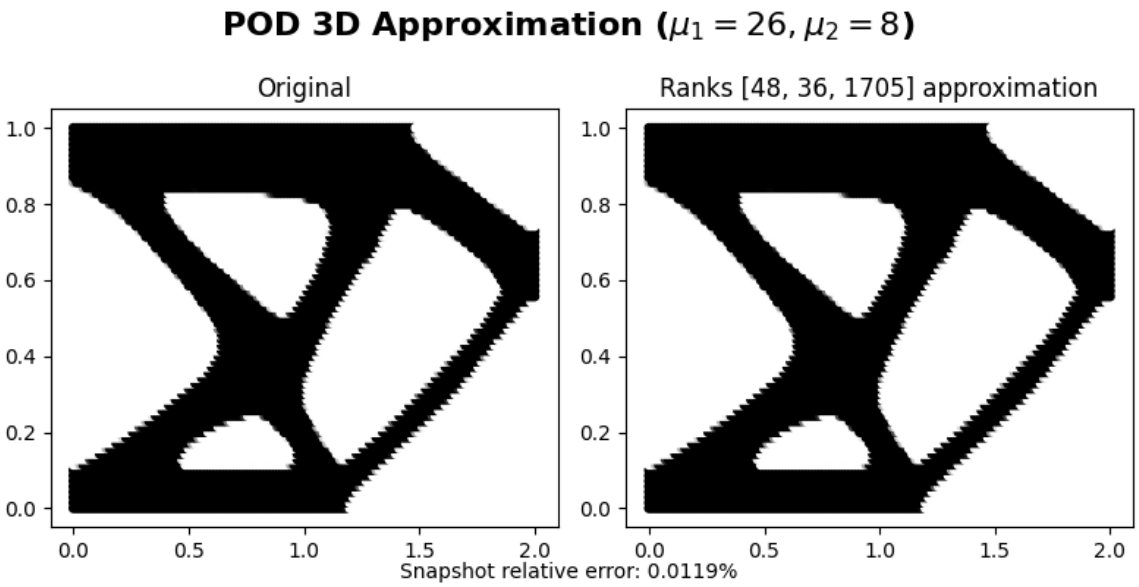
3.1.1 Results

We chose $10^{-3}, 0.1$ and 0.25 for ϵ, which yield to ranks $(48, 36, 1705), (35, 28, 1084)$ and $(21, 17, 593)$, respectively.

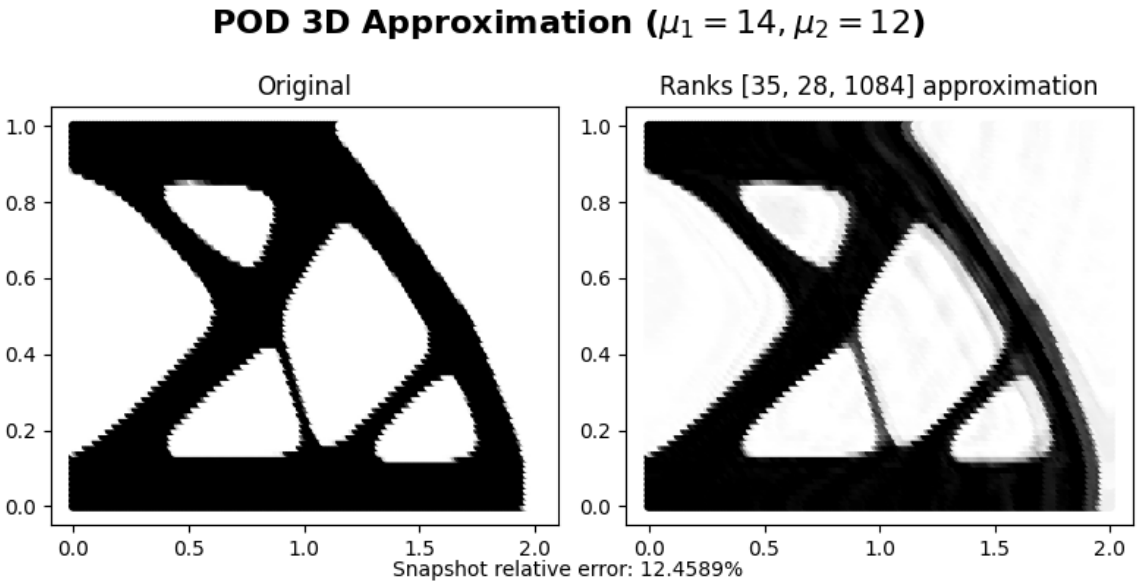
Next we will show the different reductions and the computed error for each snapshot.

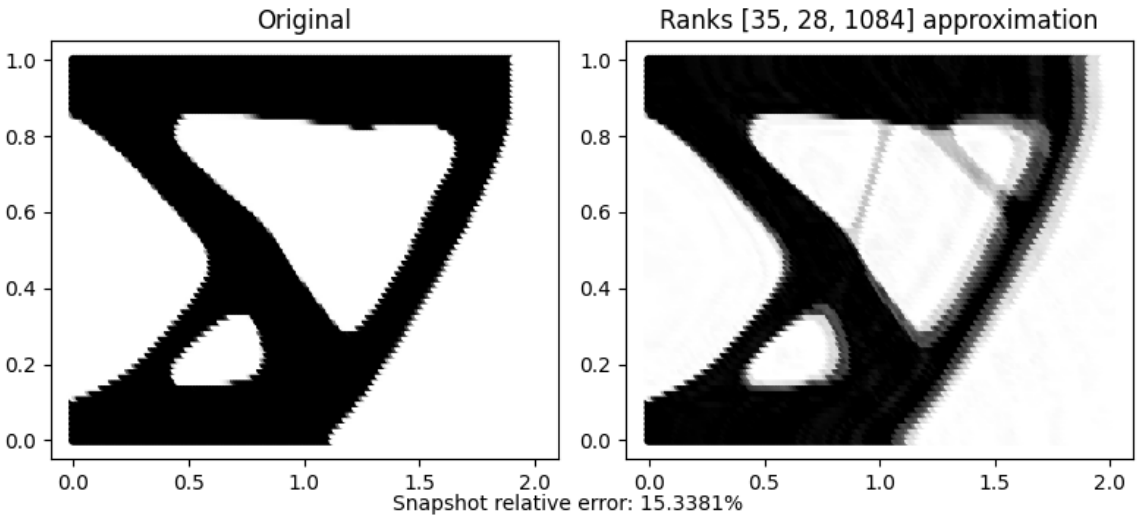
$\epsilon = 10^{-3}$:


POD 3D Approximation ($\mu_1 = 22, \mu_2 = 25$)
Original
Ranks [48, 36, 1705] approximation
Snapshot relative error: 0.0072%

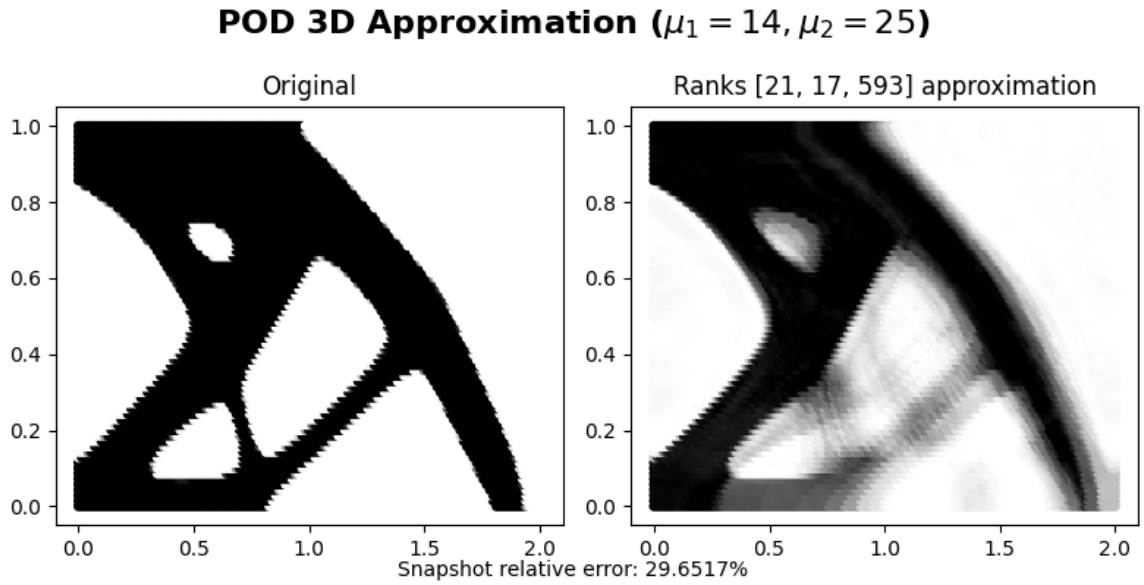

POD 3D Approximation ($\mu_1 = 26, \mu_2 = 8$)
Original
Ranks [48, 36, 1705] approximation
Snapshot relative error: 0.0119%

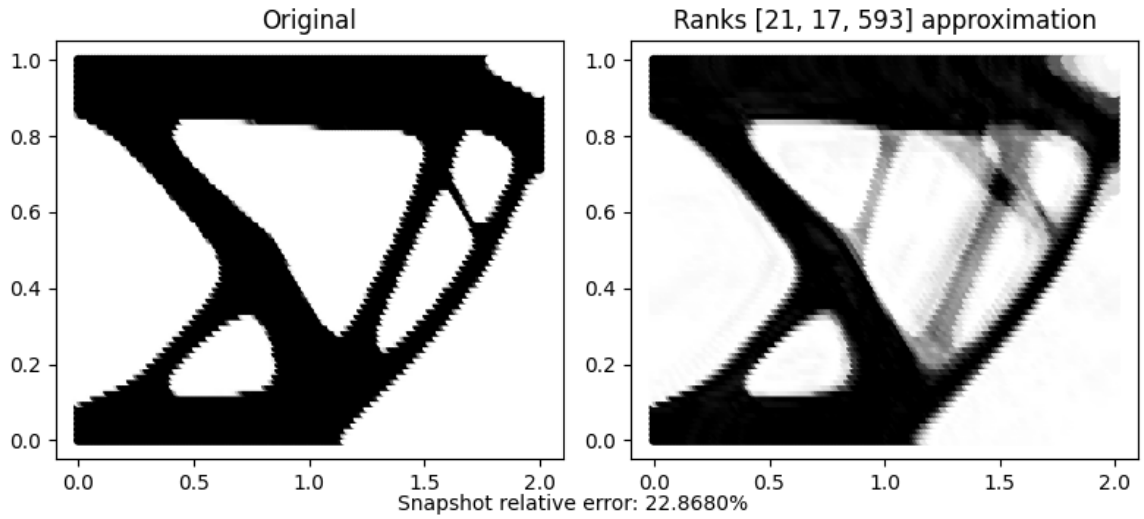
$\epsilon = 0.1$:


POD 3D Approximation ($\mu_1 = 14, \mu_2 = 12$)
Original
Ranks [35, 28, 1084] approximation
Snapshot relative error: 12.4589%


POD 3D Approximation ($\mu_1 = 34, \mu_2 = 25$)
Original
Ranks [35, 28, 1084] approximation
Snapshot relative error: 15.3381%

$\epsilon = 0.25$:


POD 3D Approximation ($\mu_1 = 14, \mu_2 = 25$)
Original
Ranks [21, 17, 593] approximation
Snapshot relative error: 29.6517%


POD 3D Approximation ($\mu_1 = 29, \mu_2 = 10$)
Original
Ranks [21, 17, 593] approximation
Snapshot relative error: 22.8680%

Here we show the errors for each snapshot. The x-axis is $s = 1, ..., T \cdot K$ the snapshot, whilst the y axis is the error described in previous slides. The first error is with the snapshots centered while the second one is the snapshots real error.

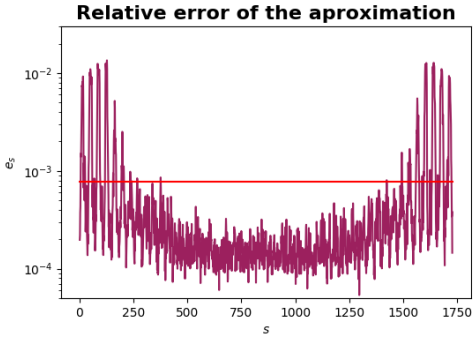


Figure 1: $\hat{e}_\epsilon$ for $\epsilon = 10^{-3}$

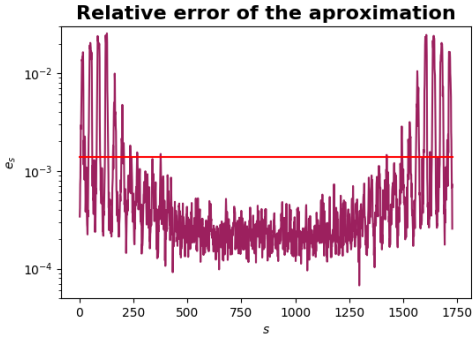


Figure 2: e_ϵ for $\epsilon = 10^{-3}$

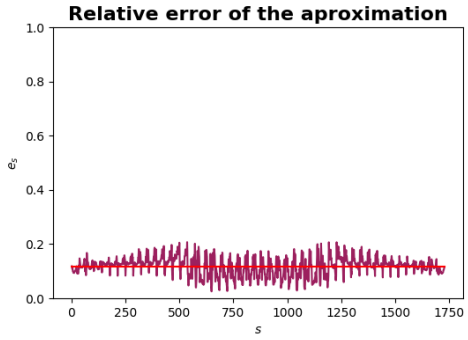


Figure 3: $\hat{e}_\epsilon$ for $\epsilon = 10^{-1}$

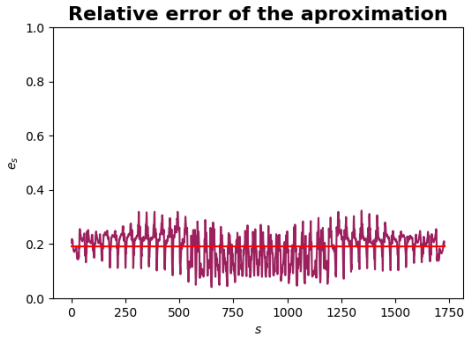


Figure 4: e_ϵ for $\epsilon = 10^{-1}$

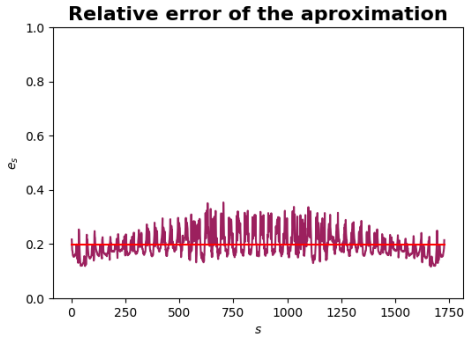


Figure 5: $\hat{e}_\epsilon$ for $\epsilon = 0.25$

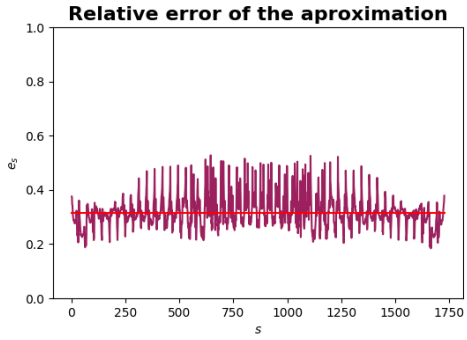


Figure 6: e_ϵ for $\epsilon = 0.25$

3.2 Optimization

Given $\mathcal{A} \in \mathbb{R}^{T\times K\times D}$, we consider $\mathbf{A} = \pi_3(\mathcal{A}) \in \mathbb{R}^{D\times(T\cdot K)}$, the mode 3 unfolding. $\mathbf{A}$ is organized as follows:

$$\mathbf{A} = \begin{pmatrix} | & & | & | & & | & & | & & | \\ \Phi_{1,1} & \dots & \Phi_{1,K} & \Phi_{2,1} & \dots & \Phi_{2,K} & \dots & \Phi_{T,1} & \dots & \Phi_{T,K} \\ | & & | & | & & | & & | & & | \end{pmatrix} \tag{7}$$

We remove $\tilde{k}$ snapshots randomly from $\mathbf{A}$, which we then aim to approximate. For a given snapshot $\boldsymbol{\mu}_{t,k}$, we approximate it using a linear combination of its neighbors, namely $\boldsymbol{\mu}_{t+1,k}$, $\boldsymbol{\mu}_{t-1,k}$, $\boldsymbol{\mu}_{t,k+1}$ and $\boldsymbol{\mu}_{t,k-1}$. In practice, the reconstruction is performed by taking the average of the available neighboring snapshots.

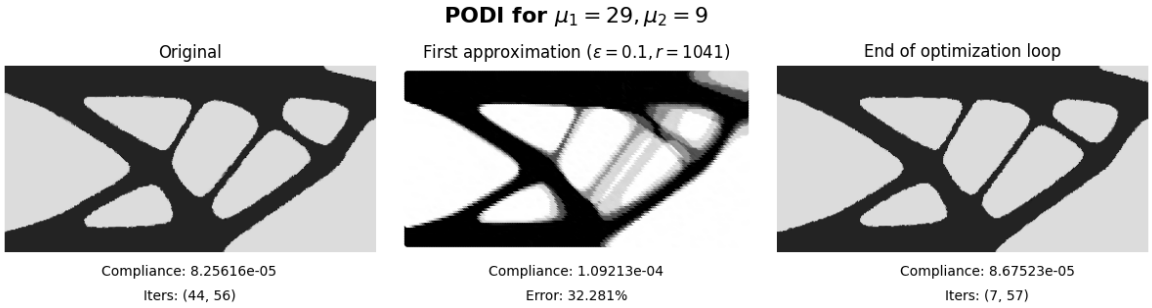
In our experiments, we set $\tilde{k} = 80$ and impose the condition that if $\boldsymbol{\mu}_{t,k}$ is removed, none of its direct neighbors ($\boldsymbol{\mu}_{t\pm1,k}$ or $\boldsymbol{\mu}_{t,k\pm1}$) are removed. This ensures that the missing snapshots are distributed more evenly across the dataset.

Once the missing data are reconstructed, we run the *FreeFEM++* optimization program using the interpolated snapshots as initial guesses. This significantly reduces the number of iterations required in the first optimization loop.

3.2.1 Results

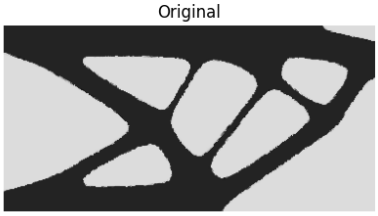
We used different values for ϵ which are detailed directly in the figures.

Note that the error shown is the compliance error between the original solution and the initial guess. That is, if we denote c_r, c_o the compliance of the original snapshot and the initial guess respectively, then $e_\epsilon = \frac{c_o - c_r}{c_r}$. There is no absolute value as the compliance of the initial guess might be lower than the real one and we capture that due to the fact that $e_\epsilon < 0$.



PODI for $\mu_1 = 29, \mu_2 = 9$

Original

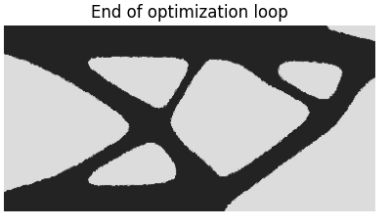


Compliance: 8.25616e-05
Iters: (44, 56)

First approximation ($\varepsilon = 0.5, r = 166$)

Compliance: 1.12364e-04
Error: 36.097%

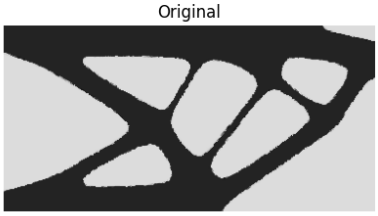
End of optimization loop



Compliance: 8.82810e-05
Iters: (10, 54)

PODI for $\mu_1 = 29, \mu_2 = 9$

Original

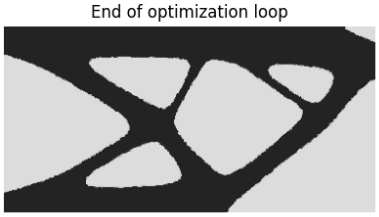


Compliance: 8.25616e-05
Iters: (44, 56)

First approximation ($\varepsilon = 0.8, r = 17$)

Compliance: 1.47825e-04
Error: 79.048%

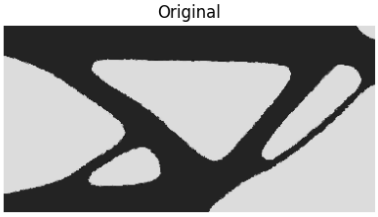
End of optimization loop



Compliance: 8.18932e-05
Iters: (12, 54)

PODI for $\mu_1 = 29, \mu_2 = 12$

Original



Compliance: 5.55880e-05
Iters: (34, 52)

First approximation ($\varepsilon = 0.1, r = 1037$)

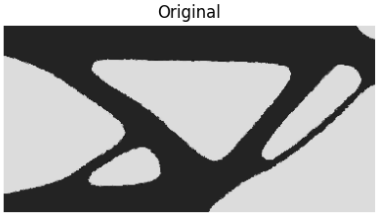
Compliance: 7.25426e-05
Error: 30.5%

End of optimization loop

Compliance: 5.77555e-05
Iters: (8, 52)

PODI for $\mu_1 = 29, \mu_2 = 12$

Original

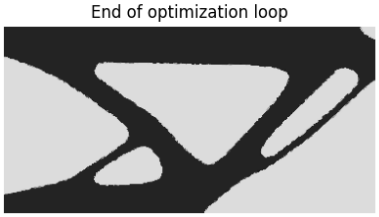


Compliance: 5.55880e-05
Iters: (34, 52)

First approximation ($\varepsilon = 0.5, r = 160$)

Compliance: 7.42114e-05
Error: 33.503%

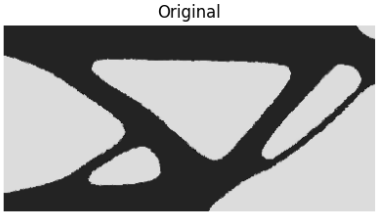
End of optimization loop



Compliance: 5.32645e-05
Iters: (10, 51)

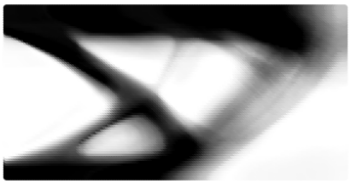
PODI for $\mu_1 = 29, \mu_2 = 12$

Original



Compliance: 5.55880e-05
Iters: (34, 52)

First approximation ($\varepsilon = 0.8, r = 16$)



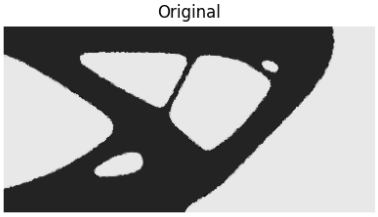
Compliance: 9.83458e-05
Error: 76.919%

End of optimization loop

Compliance: 5.68823e-05
Iters: (15, 46)

PODI for $\mu_1 = 36, \mu_2 = 22$

Original

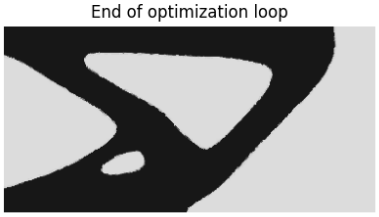


Compliance: 8.49620e-05
Iters: (34, 48)

First approximation ($\varepsilon = 0.1, r = 1037$)

Compliance: 1.04898e-04
Error: 23.465%

End of optimization loop



Compliance: 8.45644e-05
Iters: (8, 44)

PODI for $\mu_1 = 36, \mu_2 = 22$

Original

Compliance: 8.49620e-05
Iters: (34, 48)

First approximation ($\varepsilon = 0.5, r = 160$)

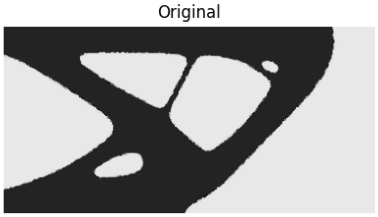
Compliance: 1.05812e-04
Error: 24.54%

End of optimization loop

Compliance: 8.10339e-05
Iters: (9, 44)

PODI for $\mu_1 = 36, \mu_2 = 22$

Original

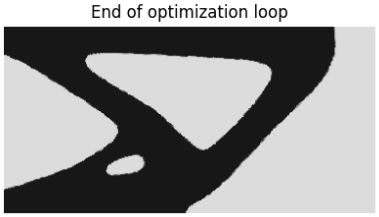


Compliance: 8.49620e-05
Iters: (34, 48)

First approximation ($\varepsilon = 0.8, r = 16$)

Compliance: 1.29032e-04
Error: 51.87%

End of optimization loop



Compliance: 9.03537e-05
Iters: (12, 43)

4 Proper Generalized Decomposition (PGD)

Proper Generalized Decomposition (PGD) is a model order reduction technique designed to efficiently approximate high-dimensional problems by exploiting a separated representation of the solution. Unlike projection-based methods such as POD, PGD constructs the reduced-order solution in an *a priori* separated form, which allows for explicit parametric dependence and avoids the curse of dimensionality in multi-parametric or time-dependent problems.

The main idea of PGD is to approximate the solution as a sum of separable functions, each depending on a subset of the problem's coordinates or parameters. For example, consider a high-dimensional solution $u(\mathbf{x}, t, \boldsymbol{\mu})$, where $\mathbf{x}$ represents spatial coordinates, t is time, and $\boldsymbol{\mu}$ denotes a set of parameters. PGD seeks an approximation of the form

$$u(\mathbf{x}, t, \boldsymbol{\mu}) \approx \sum_{i=1}^{R} X_i(\mathbf{x}) T_i(t) M_i(\boldsymbol{\mu}), \tag{8}$$

where R is the number of retained modes (rank of the decomposition), and X_i, T_i, and M_i are univariate functions capturing the spatial, temporal, and parametric dependencies, respectively.

The functions X_i, T_i, and M_i are typically computed iteratively using a greedy algorithm that adds one mode at a time by solving a sequence of lower-dimensional problems. This procedure reduces computational complexity from that of a full-order model to the sum of several lower-dimensional problems, enabling real-time or multi-query simulations in contexts such as optimization, control, and uncertainty quantification.

The main advantage of PGD lies in its ability to provide explicit parametric solutions and handle high-dimensional problems efficiently. By constructing a separated representation, PGD allows one to explore the solution manifold without reconstructing the full-order model for each new set of parameters, making it particularly useful in multi-parametric studies, design optimization, and scenarios where classical reduced-order methods may become intractable.

4.1 Reduction

For the PGD-based reduction, we employ the encapsulatedPGD toolbox developed by the LaCàN research group.

We consider the $T \times K$ snapshots stored in the *Raw* folder and assemble them into the matrix $\mathbf{A} \in \mathbb{R}^{D \times (TK)}$ where the data are centered as follows:

$$\mathbf{A} = \begin{pmatrix} | \\ \hat{\Phi}_{t,k} \\ | \end{pmatrix}, \quad \hat{\Phi}_{t,k} = \Phi_{t,k} - \bar{\Phi}, \quad \bar{\Phi} = \frac{1}{TK} \sum_{t=1}^{T} \sum_{k=1}^{K} \Phi_{t,k}. \tag{9}$$

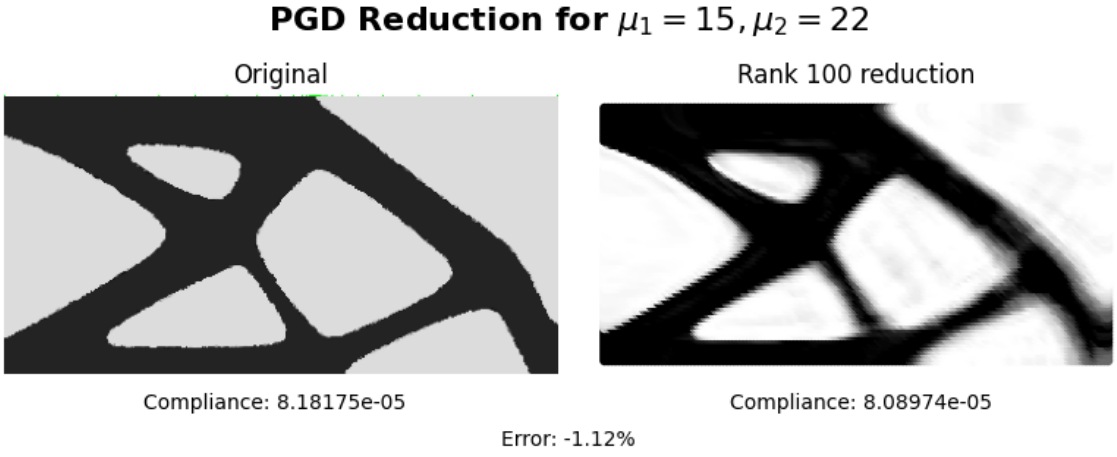
After creating the matrix $\mathbf{A}$ we create the separated tensor $\mathcal{X}$ and compress it using *epgd.compress($\mathcal{X}$)* with the following parameters:

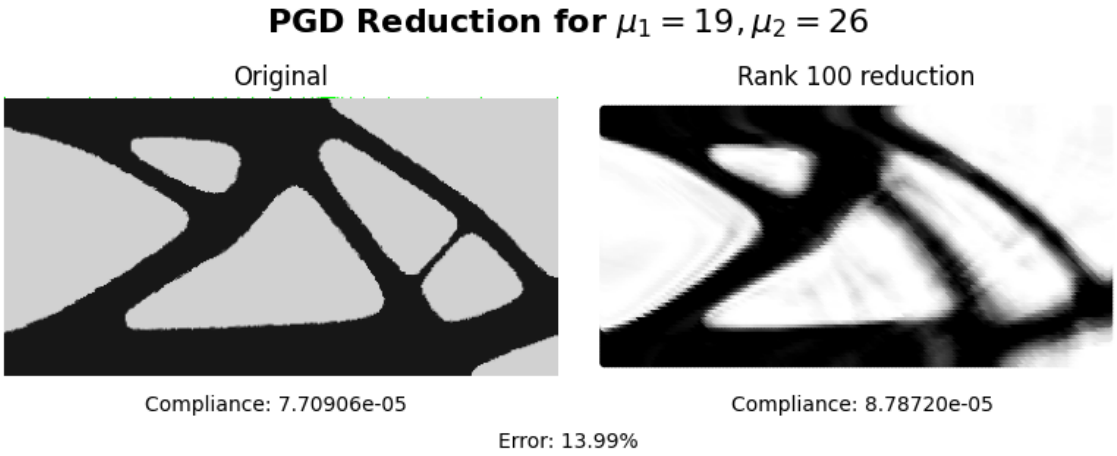
Parameter	Description	Value
$maxModes$	Maximum number of modes	100
$tolModes$	Tolerance for each mode	10^{-3}
$maxIter$	Maximum iteration steps	50
$tolIter$	Tolerance for each step	10^{-4}
$tolTruncateLastTerm1$	Truncation 1 tolerance	10^{-12}
$tolTruncateLastTermE$	Truncation E tolerance	10^{-3}

Table 2: Parameters for PGD compression used

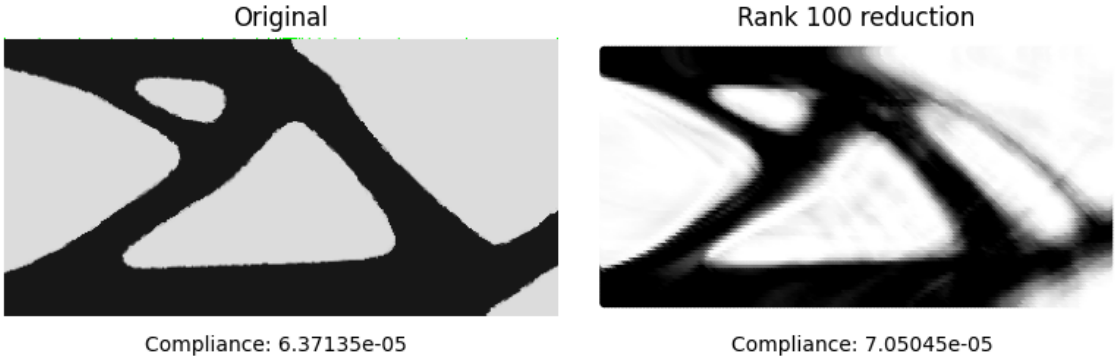
4.1.1 Results

In this section, we present several reductions obtained with PGD, along with their corresponding compliance values, and compare them to the expected solutions. The errors shown in the figures are measured with respect to the compliance.

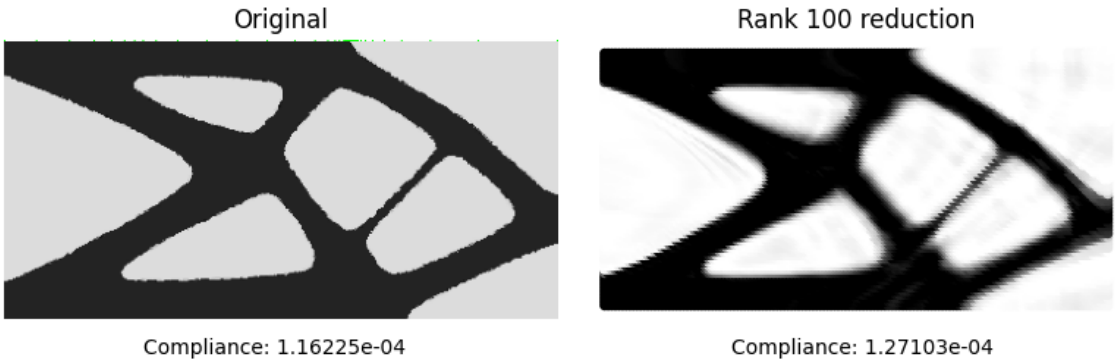




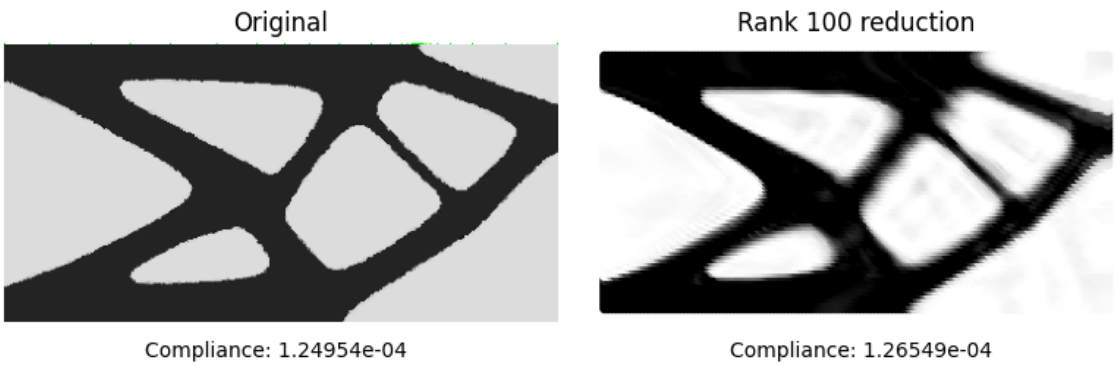
PGD Reduction for $\mu_1 = 20, \mu_2 = 25$



PGD Reduction for $\mu_1 = 21, \mu_2 = 33$



PGD Reduction for $\mu_1 = 27, \mu_2 = 1$



The following figure shows the reduction error computed for each snapshot s. It is defined as following:

$$e_s = \frac{\|\Phi_s - \Phi_s^{PGD}\|_2}{\|\Phi_s\|_2},$$

where Φ_s may be centered or uncentered.

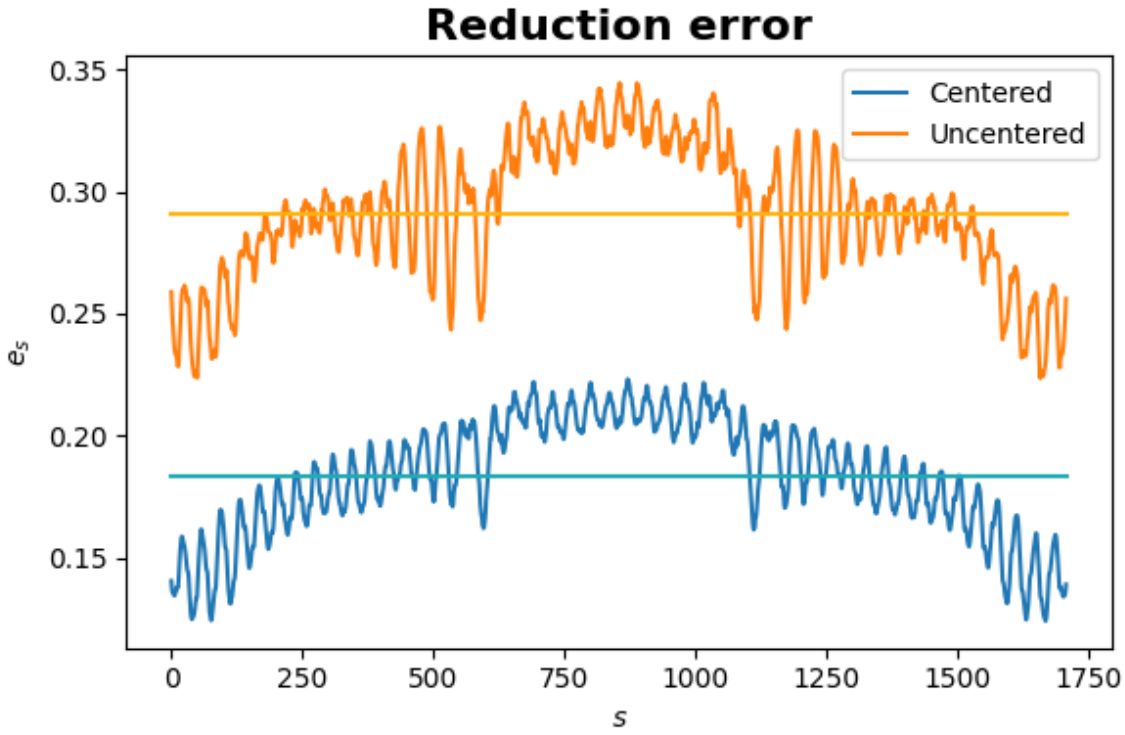


Figure 7: Errors

4.2 Optimization

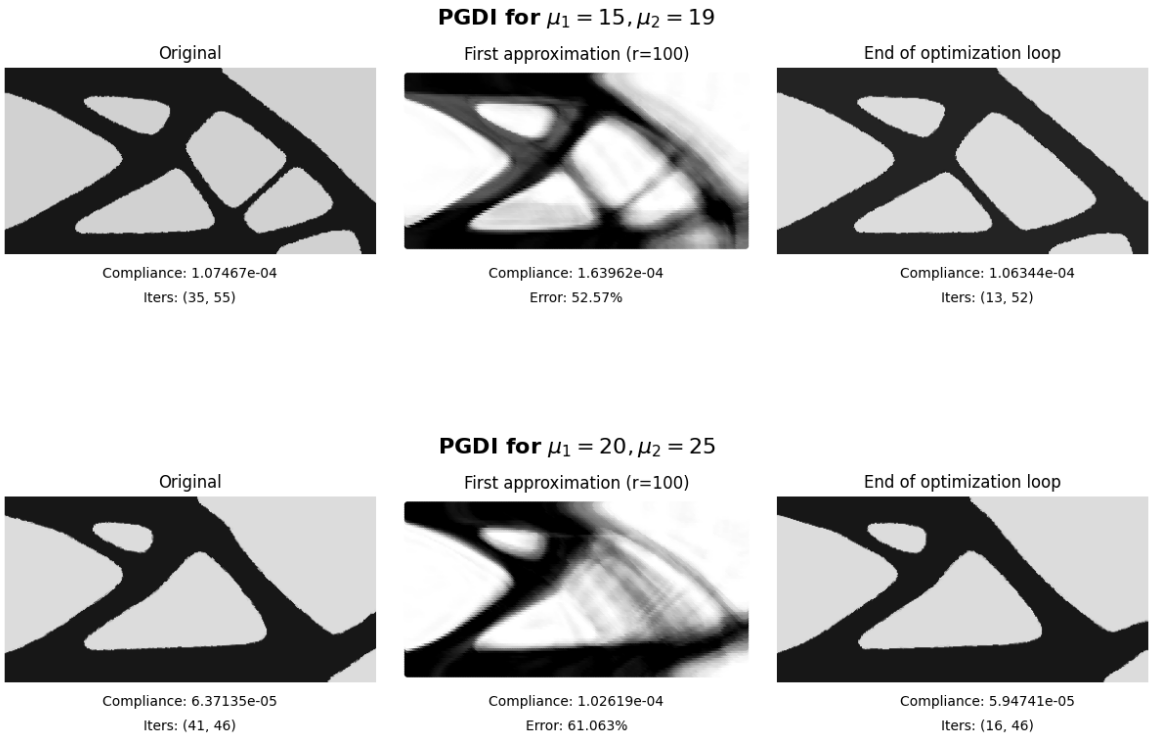
We consider the case of $\tilde{k} = 80$ unknown snapshots that we aim to approximate. Our approach begins by setting the corresponding entries in the matrix $\mathbf{A}$ to zero, since these unknown snapshots do not contribute to the initial reduction. After performing the reduction via PGD, we identify the neighbors of each unknown snapshot (for example, if $\mu = (10, 20)$, its neighbors are $(11, 20), (9, 20), (10, 19), (10, 21)$).

Next, we approximate each unknown snapshot by computing the mean of its available neighbors. If some neighbors are also unknown, they are excluded from the averaging process.

The interpolated snapshot is then used as an initial guess for the optimization procedure. We subsequently evaluate how this initialization affects both the number of iterations required for convergence and the performance of the compliance.

4.2.1 Results

Here we will show the results of both the interpolation step and the solution computed after the optimization process.



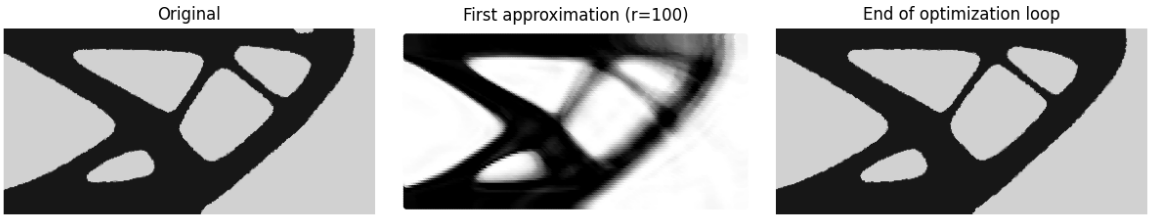
PGDI for $\mu_1 = 28, \mu_2 = 17$

Compliance: 9.42893e-06
Iters: (17, 23)

Compliance: 1.33614e-05
Error: 41.706%

Compliance: 8.87807e-06
Iters: (10, 26)

PGDI for $\mu_1 = 34, \mu_2 = 17$

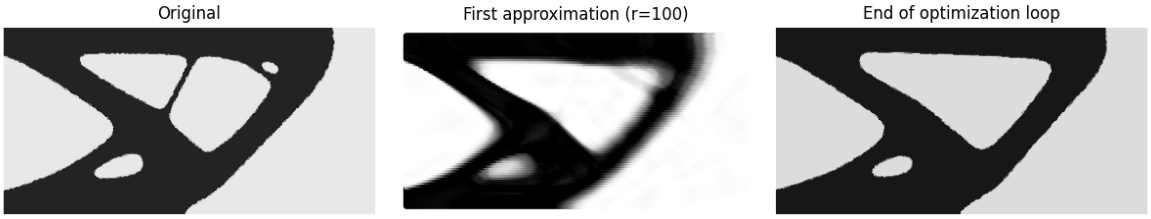


Compliance: 1.00055e-04
Iters: (35, 52)

Compliance: 1.30054e-04
Error: 29.983%

Compliance: 9.42859e-05
Iters: (10, 52)

PGDI for $\mu_1 = 36, \mu_2 = 22$



Compliance: 8.49620e-05
Iters: (34, 48)

Compliance: 1.11099e-04
Error: 30.763%

Compliance: 8.41547e-05
Iters: (10, 43)

PGDI for $\mu_1 = 7, \mu_2 = 18$

Compliance: 5.10392e-05
Iters: (35, 37)

Compliance: 5.21071e-05
Error: 2.0923%

Compliance: 4.47344e-05
Iters: (9, 39)

5 Conclusions

In this work, we have applied model reduction techniques to efficiently approximate high-dimensional problems. By relying on reduced representations, we were able to capture the essential behavior of the system while significantly lowering computational costs.

The results illustrate the strengths of reduction methods when dealing with large datasets and parametric variations. In particular, we showed that even in the presence of incomplete or missing information, simple reconstruction strategies combined with reduced models can provide accurate approximations and accelerate subsequent optimization.

More broadly, this study highlights the usefulness of reduction techniques for bridging the gap between complex numerical simulations and practical applications. By reducing the computational burden without compromising accuracy, these approaches open the door to faster analyses, more efficient optimization, and a wider exploration of parameter spaces.

6 References

References

[1] Göran Bergqvist and Erik G. Larsson. Higher-order singular value decomposition: Theory and an application. *IEEE Transactions on Signal Processing*, 2009.

[2] Vineet Bhatt, Sunil Kumar, and Seema Saini. Tucker decomposition and applications. *Materials Today: Proceedings*, 2021.

[3] Anindyas Chatterjee. An introduction to the proper orthogonal decomposition. *Current Science*, 78(7):808–817, 2000.

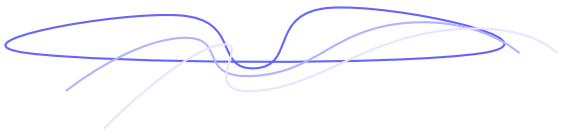
[4] Francisco Chinesta, Antonio Huerta, Gianluigi Rozza, and Karen Willcox. Model reduction methods. In *Encyclopedia of Computational Mechanics*. John Wiley & Sons, second edition, 2017.

[5] Elías Cueto, Francisco Chinesta, and Antonio Huerta. Model order reduction based on proper orthogonal decomposition. In Francisco Chinesta and Pierre Ladevèze, editors, *Separated Representations and PGD-Based Model Reduction: Fundamentals and Applications*, CISM International Centre for Mechanical Sciences, pages 1–46. Springer, Vienna, 2014.

[6] Elías Cueto, David González, and Iciar Alfaro. *Proper Generalized Decompositions: An Introduction to Computer Implementation with Matlab*. SpringerBriefs in Applied Sciences and Technology. Springer International Publishing, 2016. Extras online at http://extras.springer.com.

[7] Lieven De Lathauwer, Bart De Moor, and Joos Vandewalle. A multilinear singular value decomposition. *SIAM Journal on Matrix Analysis and Applications*, 21(4):1253–1278, 2000.

[8] Julien Weiss. A tutorial on the proper orthogonal decomposition. In *AIAA Aviation Forum*, Dallas, Texas, United States, June 2019. American Institute of Aeronautics and Astronautics. Accepted manuscript (Postprint), licensed under CC BY 4.0.



Miquel P. Baztán Grau
Universitat Politècnica de Catalunya